

Security - Application Security II - Defense - prevent, detect and kill attacks

2026-04-16 19:26

Status: #idea

Tags: Security

Security - Application Security II - Defense - prevent, detect and kill attacks

Welcome to Lecture 7! Don't worry about missing it; application security defenses are complex, but we'll break down every single slide and answer all of your questions.

As a roadmap, this lecture shifts from the **Attacker's Perspective** (how to break things) to the **Defender's Perspective** (how to stop the attacks).

To answer your overarching question: **Every defense we discuss will fall into one of three categories:**

1. **Preventive:** Makes the exploit impossible to execute.
2. **Detective:** Notices the exploit is happening and kills the program.
3. **Reactive/Corrective:** Fixes the damage after the fact (we mostly focus on the first two).

Let's dive in.

Part 1: Recap & Attacker Steps (Slides 1-7)

Recap: buffer overflow to inject attacker code

Diagram illustrating a buffer overflow to inject attacker code. The memory layout shows a buffer, old rip (8 bytes), and previous function stuff. The buffer is filled with '0x00' and '0x01' bytes. The 'rip' register points to the 'old rip (8 bytes)' field, which is overwritten with '0x12345678901234567890'. This causes the program to execute attacker code instead of the original function stuff.

Recap: buffer overflow to affect logic

Diagram illustrating a buffer overflow to affect logic. The memory layout shows a buffer, authenticated, old rip (8 bytes), and previous function stuff. The buffer is filled with '0x00' and '0x01' bytes. The 'authenticated' variable is overwritten with '0x00000000'. This causes the program to skip the 'if (authenticated)' check and execute the rest of the function.

Recap: other attacks	Attacker steps
• Heap smashing/overflow: similar to stack smashing/overflow • Different targets, can have different impacts • Format string vulnerabilities • Unsanitized user input interpreted as formatting directives • Unexpected formatting directives make program behave wrongly • Integer over/underflow: numbers in computers are not infinite! • Attackers affects program logic by carefully choosing values • Time-Of-Check Time-Of-Use vulnerability: "race condition" between checking and using property	As an attacker, to exploit a vulnerable program: • Find some code that we want to execute (but we should not be able to) • If not exists, inject our own code as binary instructions • Access memory beyond theoretical limits (overflow) to modify program status to execute that code • If jumping: need to guess code's address • Usage of NOP sleds allows for imperfect guesses • Profit

Security - Application Security II - Defense - prevent, detect and kill attacks - image-1.webp

Attacker Steps Explained:

- *"If not exists, inject our own code"*: Sometimes the program already contains the code the attacker wants (like a function that grants admin access). If it doesn't, the attacker must write their own binary assembly instructions (shellcode) and inject them into the stack.
- *"If jumping, need to guess code's address"*: You are correct that the CPU *always* jumps to the return address. But the attacker is the one *overwriting* that address. To make the CPU jump to the injected shellcode, the attacker must overwrite the return address with the exact memory address of where their shellcode lives. That memory address often has to be guessed.

What is a buffer size?

Yes, a `char` in C is exactly 1 byte. So `char buffer[4]` allocates exactly 4 bytes of memory.

Reminder:

Heap Overflows

Heap overflow

- Until now, we have been doing “**stack smashing**”
- ... but programs use another memory region: the **heap**!
 - Memory reserved via `malloc`
 - Memory used to survive beyond a function's end
 - Many high-level languages (Java, Python) use the heap a lot for object-oriented programming

```
void f() {  
    char* buf = (char*) malloc(4);  
    char* buf2 = (char*) malloc(42);  
    char* user_input = /*read user input*/  
    strcpy(buf, user_input);  
    // Rest of the function  
}
```

- If user input > 4, then overwrite heap
- It might be `buf2`, it might be something unrelated!

As mentioned earlier, the Heap is dynamic memory used by `malloc()`.

```
void f() {  
    char* buf = (char*) malloc(4);  
    char* buf2 = (char*) malloc(42);  
    char* user_input = /*read user input*/  
    strcpy(buf, user_input);  
}
```

If we overflow `buf` here, we aren't on the stack anymore, so there is no Return Address to overwrite. Instead, we spill into whatever data is sitting next to us on the heap (like `buf2`, or sensitive object metadata in languages like Java or C++). By corrupting this data, we can change the internal logic of the program.

So consequences:

- Overflow into adjacent objects
- In object-oriented programming: overflow into vtables (class information)
⇒ Change object methods!
- Overflow heap metadata itself

Part 2: Preventing Exploits (Slides 8-13)

Remember

1. **Preventive:** Makes the exploit impossible to execute.
2. **Detective:** Notices the exploit is happening and kills the program.
3. **Reactive/Corrective:** Fixes the damage after the fact (we mostly focus on the first two).

Security - Application Security II - Defense - prevent, detect and kill attacks - image-4.webp

A code audit is a comprehensive, systematic examination of a software program's source code, architecture, and security to identify bugs, vulnerabilities, and inefficiencies. It provides an unbiased, third-party assessment to ensure the software is secure, maintainable, and compliant with best practices, ultimately reducing technical debt and supporting long-term performance.  Wikipedia +4

Security - Application Security II - Defense - prevent, detect and kill attacks - image-5.webp

To **prevent** exploits:

- Make exploitation **steps** more **difficult or impossible**
- **Avoid bugs** via secure coding and better **code audit**
 - Automated tools, limit to subsets of C, ...
- **Improve architecture:** improve OS, libraries, compiler, ...
 - Make exploits of *all* programs more difficult

⇒ These strategies are **complementary** and are all widely used

Security - Application Security II - Defense - prevent, detect and kill attacks - image-3.webp

Why circle "Improve Architecture"?

We can try to fix bugs by writing better code ("secure coding"), but humans are flawed and will always make mistakes. Improving the architecture (the OS, the compiler, the CPU) is circled because it provides a safety net that protects **all** programs automatically, even if the programmer made a mistake. These strategies are **complementary**, meaning we don't just pick one; we use them all together like layers of armor.

Defense 1: Non-Executable Memory (NX Bit / W^X)

Non-executable memory

Attacker steps

Inject our own instructions as binary on the stack. Overwrite `rip` so that execution moves to the stack.

Defences?

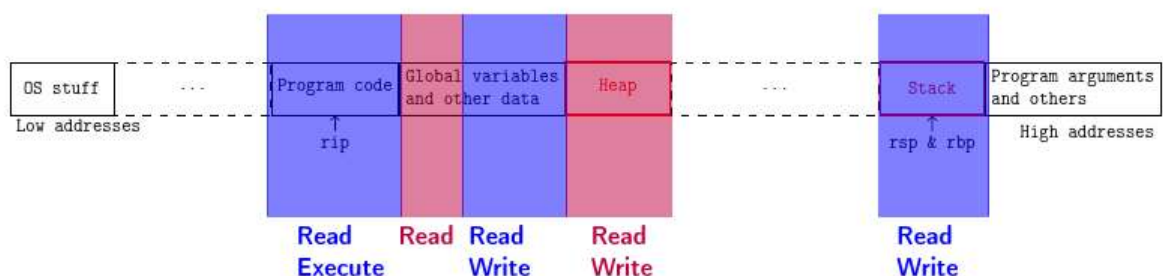
- In modern systems CPU does not access memory directly → Memory Management Unit (MMU)
 - Or Memory Protection Unit (MPU) for smaller systems
- OS can mark stack and heap **segments**¹ as *non-executable*
- CPU refuses to execute instructions from non-executable segments
 - **Segmentation fault**

¹Modern systems use *pages* of memory and not *segments*, but you can think of a segment as a sequence of pages.

Security - Application Security II - Defense - prevent, detect and kill attacks - image-6.webp

Category: **PREVENTIVE**

- **The MMU Recap:** The Memory Management Unit (MMU) is hardware inside the CPU. It translates "virtual memory" (what your program sees) into "physical memory" (the actual RAM chips). It also enforces permissions.
- **Permissions (Read/Write/Execute):** The OS assigns permissions to different memory segments.
 - Code segment: **Read & Execute** (Blue in the slide). You can read the instructions and run them, but you can't overwrite them.
 - Stack/Heap segment: **Read & Write**. You can store data here, but you are not allowed to execute it. So NO shellcode in the stack that can be executed!
- **Why can attackers still overwrite the stack?** NX (non executable) memory prevents *executing* the stack, but it does NOT prevent *writing* to it! Buffer overflows are fundamentally a "write" operation. The program will happily let you overwrite the return address. The defense kicks in (Segmentation Fault) only when the CPU actually *tries to run* your injected shellcode.



Security - Application Security II - Defense - prevent, detect and kill attacks - image-7.webp

Bypassing NX (non exec) Memory: Return-Oriented Programming (ROP)

If the stack is non-executable, the attacker cannot inject their own code. They can not do that easy shellcode inject and jump ret address to that place on the stack, it's NX there. So attackers gotta use only whats in the executable regions. And maybe frankenstein/build malicious code from that? So, they use **ROP**.

Return-oriented programming (ROP)

Does non-executable memory solve everything? What can it *not* solve?

- Attacker can still overwrite stack content if buffer overflow available
 - In particular, return addresses
- Attacker can "frankenstein" their payload from existing instructions in executable memory
- libc (standard C library) particularly juicy target: **return-to-libc attacks**
 - e.g. contains function to execute command (`system`), contains string `"/bin/sh"`
 - Contains all sort of functions with all sorts of instructions: e.g. loading register value from stack
 - **Has well-known structure at well-known addresses**

Security - Application Security II - Defense - prevent, detect and kill attacks - image-8.webp

Instead of jumping to the stack, the attacker overwrites the return address to jump to code that is *already inside the executable code segment*. Specifically, they target `libc` (the standard C library, which contains functions like `system("/bin/sh")` to open a terminal). They "frankenstein" an exploit by stitching together pieces of code that already exist. ("Standard stack smashing" refers to the old method of injecting shellcode). If we know the memory address where libC starts, and we know that from that point, if we do +50 addresses there is a 'sudo rm -rf /home/aryan/.steam/CSGO', the attacker can find the libC memory address and just return to that address + 50.

Defense 2: ASLR (Address Space Layout Randomization)

Category: **PREVENTIVE**

To stop ROP, the OS uses ASLR. Every time you run a program, ASLR scrambles the memory locations of the stack, heap, and libraries (like `libc`).

ASLR	ASLR caveats
Attacker steps Exploit the fact that libc is loaded at well-known address and jump to it. Defences? Address Space Layout Randomization (ASLR): place standard libraries and other useful elements randomly in memory - Prevents return oriented programming (and return-to-libc in particular) - Prevents guessing return address in "standard" stack smashing - NOP sled of little help if random space too big Currently deployed on all OSs	- Only randomizes start positions, not contents - To be applied to program instructions, compiler needs to support it - All major compilers and OSs should support it nowadays - Needs enough randomness in the possible addresses - Should not be a problem with 64-bit address space - Needs to actually be random - Attackers can try to reduce possibilities with information leak (e.g. printf format vulnerabilities) - Memory alignment requirements can reduce possibilities

Security - Application Security II - Defense - prevent, detect and kill attacks - image-9.webp

To understand this, remember the context:

1. What is Return-Oriented Programming (ROP)?

Because modern operating systems use **Non-Executable (NX) memory** to mark the stack as "Read/Write but NOT Execute", attackers can no longer inject their own shellcode into a buffer and run it. The CPU will flat-out refuse.

So, attackers pivoted to ROP. Just like you described, they "Frankenstein" malicious code together.

- Instead of injecting *new* code, the attacker searches through the program's existing executable memory (like `libc`, the massive library that contains every basic C function).
- They look for tiny, harmless snippets of code that end in a `return` instruction. These snippets are called "**gadgets**".
- The attacker overflows the buffer and overwrites the stack with a long chain of return addresses, each pointing to a different gadget.
- The CPU jumps to Gadget 1, executes it, hits the `return`, pops the *next* address off the attacker's fake stack, jumps to Gadget 2, and so on.

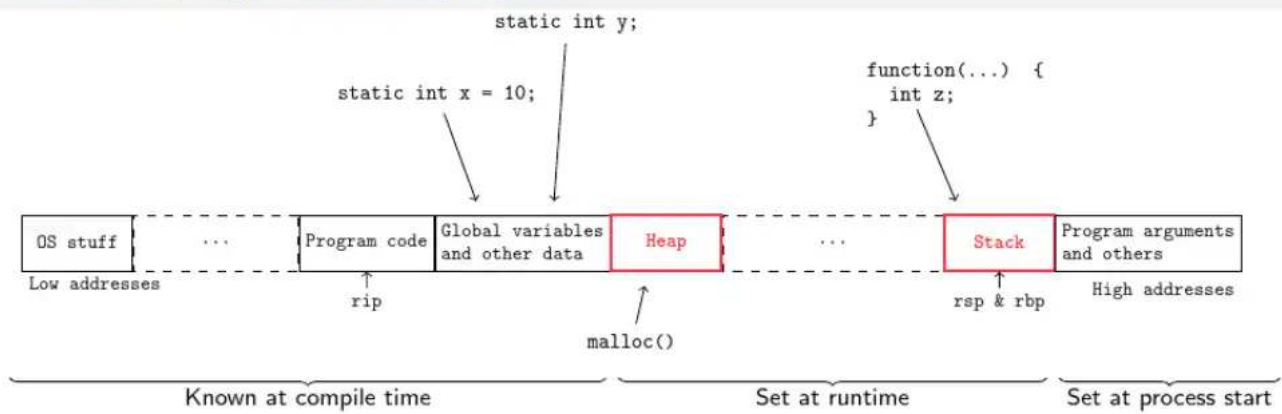
By stringing together enough harmless gadgets, the attacker can execute completely malicious logic without ever writing a single line of their own executable code.

2. How does ASLR actually work? What gets randomized?

You asked if ASLR randomizes the internals of the regions (adding jumps) or just the starting offsets.

Let's look back at the Program Memory Layout

Reminder: program memory layout



Security - Application Security II - Defense - prevent, detect and kill attacks - image-10.webp

ASLR only changes the starting offset (the base address) of the major memory regions. Think of the computer's memory like a massive, empty highway.

- **Without ASLR:** The Stack is always built at mile marker 100. The Heap is always at mile marker 20. The `libc` library is always at mile marker 50. Because it's always the same, an attacker can write an exploit on their home computer, hardcode "mile marker 50" into their payload, and it will work perfectly on your computer.
- **With ASLR:** Every single time the program starts, the OS rolls a giant pair of dice. Today, the Stack starts at mile marker 8,492. The Heap starts at 3,111. `libc` starts at 9,999,021.

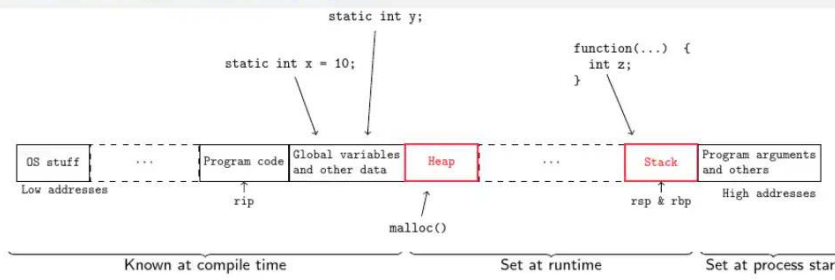
Crucially, the *internal structure* of these regions does not change. * The distance from `buffer` to the `return address` inside a stack frame is exactly the same. So if they know a start of a region, the offset is the same, so we need to prevent that the attacker can know the start address of the region.

- The distance from the start of `libc` to the `system()` function inside `libc` is exactly the same.
- Only the absolute starting position of the entire block has moved.

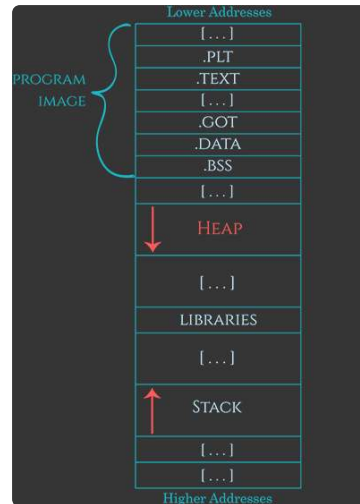
Where do libraries live?

Look at the memory layout diagram. Do you see the large `...` gaps between the Code, Heap, and Stack? Those gaps represent vast amounts of empty virtual memory. Standard libraries like `libc` are loaded into a dedicated "Memory Mapping" segment right in the middle of that empty space between the Heap and the Stack.

Reminder: program memory layout



Security - Application Security II - Defense - prevent, detect and kill attacks - image-10.webp



Security - Application Security II - Defense - prevent, detect and kill attacks - image-11.webp

3. Guessing Return Addresses & Why NOP Sleds Fail

You asked: "What do they mean guessing return address? The return address of what?"

The slide is referring to **guessing the address you want to overwrite the return address WITH.**

In a standard stack smashing attack (where you inject shellcode into a buffer), your goal is to overwrite the old return address with **the memory address of your shellcode**. You need the CPU to jump to your buffer.

- **Without ASLR:** You run the program in a debugger (like GDB), look at the memory, and see your buffer is at `0x7fffffffef000`. Great! You overwrite the return address with `0x7fffffffef000`, and you win.
- **With ASLR:** Your buffer is at `0x7fffffffef000` today. Tomorrow it's at `0x7ffcd3a8b000`. On the victim's server, it's at `0x7ffed1b2c000`. You have absolutely no idea what address to write over the return address because the entire stack has been randomly shifted.

So you can't know what to inject into the buffer, because you need to hardcode an address.

That sucks for the attacker. But hmmm what if the attacker can read memory to find out

start locations and from there do the offsets to calculate what addresses to inject in the buffer dynamically? Hmmmmmmmmm let's think about that.



Security - Application Security II - Defense - prevent, detect and kill attacks - image-12.webp

Why does a NOP Sled fail here?

You are completely correct that a NOP sled makes your target bigger. If you put 10,000 NOPs in front of your shellcode, you don't need to guess the exact address of your shellcode—you just need to guess an address *anywhere* within those 10,000 NOPs.

However, A modern 64-bit computer has an unfathomably large amount of memory addresses—**18 quintillion** (1.8×10^{19}). ASLR means, it could be anywhere. Hardcoding is no option

Even if you build a massive NOP sled of 100,000 bytes, trying to randomly guess where that 100,000-byte sled landed inside a space of 18 quintillion addresses is mathematically impossible. The "random space" (the vast emptiness of the 64-bit highway) is simply too big for your sled to cover a meaningful percentage of it.

- **Why does a NOP sled fail here?** A 64-bit computer has 1.8×10^{19} possible memory addresses. Even a massive NOP sled is like a grain of sand in the ocean. The attacker can't guess where it is.
- **Why does the compiler need to support it?** The compiler must compile the program as "Position Independent Code" (PIC). If the compiler hardcodes exact memory addresses into the program, ASLR breaks the program.
- **Info Leaks:** Because ASLR usually randomizes the *start* of the library but keeps the internals in the same order, if an attacker uses a `printf` format string bug to leak just *one* memory address, they can do the math to figure out exactly where the rest of `libc` is hiding.

☹️ Only randomizes start positions, not contents

☹️ To be applied to program instructions, compiler needs to support it

👉 All major compilers and OSs should support it nowadays

☹️ Needs enough randomness in the possible addresses

👉 Should not be a problem with 64-bit address space

- Needs to actually be random

☹️ Attackers can try to reduce possibilities with information leak (e.g. `printf` format vulnerabilities)

☹️ Memory alignment requirements can reduce possibilities

Security - Application Security II - Defense - prevent, detect and kill attacks - image-13.webp

Part 3: Detecting Exploits & CFI (Slides 14-26)

Defense 3: Stack Canaries

1. **Preventive:** Makes the exploit impossible to execute.
2. **Detective:** Notices the exploit is happening and kills the program.
3. **Reactive/Corrective:** Fixes the damage after the fact (we mostly focus on the first two).

Security - Application Security II - Defense - prevent, detect and kill attacks - image-4.webp



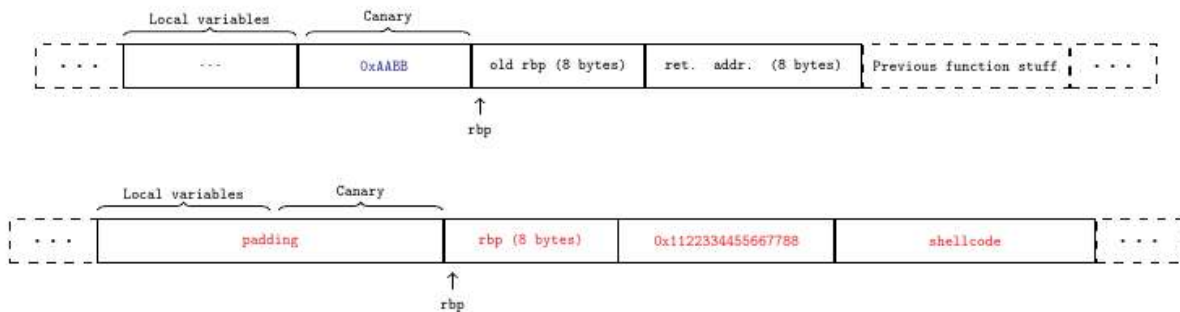
Security - Application Security II - Defense - prevent, detect and kill attacks - image-14.webp

Category: **DETECTIVE**

Named after the "canary in a coal mine" used to detect toxic gas, a stack canary is a secret value placed between your local variables and the return address.

Stack canaries

A **canary** is an alert system that fails early before more important systems fail.



- Compiler adds code to every function to check canary value before loading return address
- If loaded canary does not have expected value: immediately abort

Security - Application Security II - Defense - prevent, detect and kill attacks - image-15.webp

- **How does it not get overwritten?** The *value* of the canary is on the stack (and gets overwritten during a buffer overflow). However, the program *code* (machine code, read-execute only, in an entirely different block) that checks the canary lives in the `.text` segment, which is read-only and far away from the stack. The attacker cannot overwrite the checking code. When the function finishes, the read-only checking code looks at the stack. If the canary value is modified, the program aborts.
 - `IFF (value(MEM{RBP - 8}) == value(CANARY in register (or in .data idk)))`
-JUMP (return addr on stack)

What should we set the canary value to?

- Definitely not a well-known value that the attacker can easily inject!
- **Terminator canaries:** contain null or other (e.g. `\n`) bytes that can not be injected via classic vulnerable functions (e.g. `strcpy`, `scanf`)
- **Random canaries:** generated at runtime for each process.
 - True value stored somewhere else than stack.
 - Default for gcc when using canaries (default if "sensitive" functions called)
- **Random XOR canaries:** random value XOR-ed with control flow info (e.g. return address mask)
 - Protects against attacker targeting saved rip directly via non-buffer-overflow pointer manipulation

Security - Application Security II - Defense - prevent, detect and kill attacks - image-16.webp

- **Terminator Canaries (Null bytes):** A null byte (`\0`) tells a C string, "Stop copying here." If the canary contains a null byte, a function like `strcpy()` will hit the canary, think the string is over, and stop copying—making it mathematically impossible to overflow past the canary to the return address! So it's just so the attacker can't inject the canary in his payload to trick the system. The buffer overflow doesn't care what it overwrites.
- **Random Canaries:** The true, random value is generated when the program starts and stored in a secure CPU register or Thread Local Storage—places a standard buffer overflow cannot reach. The checking code compares the stack canary against this hidden true value.
- **XOR Canaries:** If an attacker has an advanced bug that lets them write directly to the return address *without* overflowing the buffer (skipping over the canary entirely), so the canary on the stack stays intact, they only alter the return address. an XOR canary fixes this. It cryptographically binds the canary to the return address. If you alter the return address, the math no longer matches, and the program aborts.

1. Terminator Canaries

- **Your logic:** The attacker has to put the exact canary value into their payload so the system doesn't trigger the alarm. But if that exact value contains a null byte (`\0`), `strcpy` will think the string is finished and stop copying entirely, meaning the attacker can't overwrite the return address.
- **Verdict: You are 100% correct.** That is exactly how it works. It uses the very nature of C-strings against the attacker.

2. Random Canaries

- **Your question:** Isn't comparing the stack to a hidden true value what *all* canaries do? How is this different from a Terminator canary?

- **The Answer:** Yes, the *checking mechanism* is exactly the same. The difference is **predictability**. A Terminator canary is usually static (e.g., it is always `0x000AFFFF`). If an attacker finds a different vulnerability that *doesn't* stop at null bytes (like a custom memory-copy function), they can just hardcode `0x000AFFFF` into their payload and bypass the check. A Random canary is a completely unpredictable 64-bit number that changes every time the program opens. The attacker can't hardcode it because they don't know what it is.

3. XOR Canaries & "Pointer Manipulation"

- **Your logic:** It's like a hash. The correct value relies on both the canary and the return address. If you change the return address but leave the canary intact, the math breaks.
- **Verdict: Spot on.** Now, let me explain *why* an attacker would leave the canary intact, which answers your question about pointer manipulation.

What is "Non-buffer overflow pointer manipulation"?

Think of a standard buffer overflow like a **bulldozer**. To get to the return address, it has to smash through everything in its path, including the buffer and the canary.

However, some advanced bugs (like the `%n` format string vulnerability from the previous lecture) act like a **sniper rifle**. They allow an attacker to say, "*Write my shellcode address EXACTLY at the memory location of the Return Address.*" Because it's a sniper shot, the bulldozer never runs over the canary. The canary sits on the stack, completely untouched and perfectly valid.

How the XOR Canary ("Return Address Mask") catches the sniper:

To catch this, the system entwines the canary and the return address using math.

1. When the function starts, the OS takes the `Random Canary` and `XOR`s it with the `Real Return Address`. It stores this combined "Masked" value in the secure CPU register.
2. The attacker uses a sniper bug to change the return address to `Shellcode Address`. The canary is untouched.
3. When the function finishes, the checking code takes the untouched canary from the stack and `XOR`s it with the new `Shellcode Address`.
4. Because the address changed, the math outputs garbage. It does *not* match the combined "Masked" value in the secure register. The program realizes the return address was tampered with and aborts!**

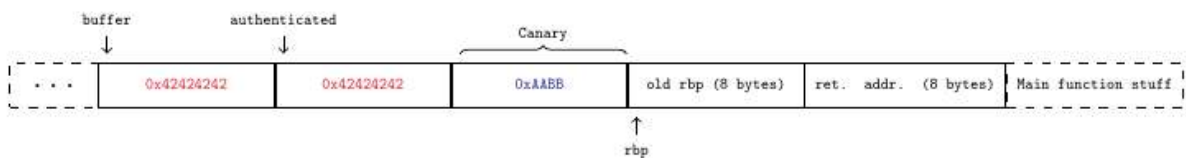
- **Overwriting Memory (%n)**

- **What they type:** 100% no way! (which triggers % n)

- **What happens:** %n is incredibly dangerous. Instead of reading data, it writes data. It counts how many characters have been printed so far, grabs the next value on the stack (treating it as a memory address), and *writes that count into that memory address*.
- **The Misuse:** An attacker can use this to rewrite the program's variables while it's running! By carefully padding the output with blank spaces before the %n , they can control the exact number that gets written. They can use this to overwrite security checks (changing an is_admin = 0 to is_admin = 1) or hijack the program entirely.

Security - Application Security II - Defense - prevent, detect and kill attacks - image-17.webp

- **The Caveat:** The canary is right next to the return address. An attacker can still overflow a small buffer and overwrite a local variable (like the `authenticated = 0` example) without ever touching the canary! To prevent this, you would have to put a canary next to *every single variable*, and check the canary not only for JUMP, but for everything involving a variable, which would devour memory and make the program painfully slow.



☹ Does not protect *all* variables against overwriting: just return address (and saved rbp)

- Would need canary after *every* memory location → uses more memory
- Would need code to check after every memory assignment → takes longer

☹ Read overflow could allow attacker to read canary value

- 👉 Increases steps that attacker needs to take and vulnerabilities that need to be present

Security - Application Security II - Defense - prevent, detect and kill attacks - image-18.webp

The attacker could still find out the current canary (read exploit with printf) and maybe the XOR/HASH rules to calculate the canary and do the sniper inject (printf with %n). But the starts must allign for the attacker.

Defense 4: Control-Flow Integrity (CFI)

Behaviour-based detection	Control Flow Integrity (CFI)
? How can we detect an ongoing attack? 🔍 If a program deviates from its expected behaviour, it might be compromised ⚠️ What is expected behaviour? ⚠️ What is deviation? ⚠️ How to check in a way that is difficult to compromise?	• A function consists of basic blocks (pieces of code) • A program execution is made of control transfers among basic blocks • Here: only look at control transfer between functions • Connections between basic blocks can be modeled as Control Flow Graph (CFG) • Expected behaviour follows CFG

Security - Application Security II - Defense - prevent, detect and kill attacks - image-19.webp

Category: **DETECTIVE**

CFI ensures the program only takes paths the programmer intended.

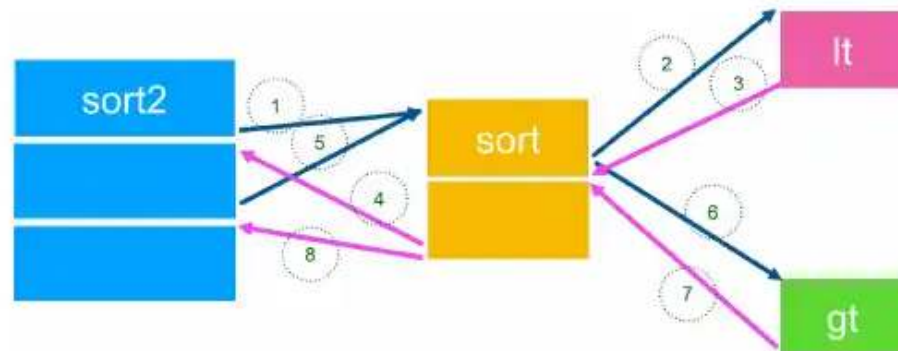
Call graph

```
void sort(int x[], int len, /*function*/
→ f) {
    // Omitted
    // Calls f at some point(s)
}

int sort2(int a[], int b[], int len) {
    sort(a, len, lt);
    sort(b, len, gt);
}
```

```
bool lt(int x, int y) {
    return x < y;
}

bool gt(int x, int y) {
    return x > y;
}
```



Security - Application Security II - Defense - prevent, detect and kill attacks - image-21.webp

- **Expected Behavior & CFG:** The compiler analyzes the code and builds a Control Flow Graph (CFG)—a map of all legal paths the program can take.

Control Flow Integrity: compliance with CFG

To check whether execution is **compliant**:

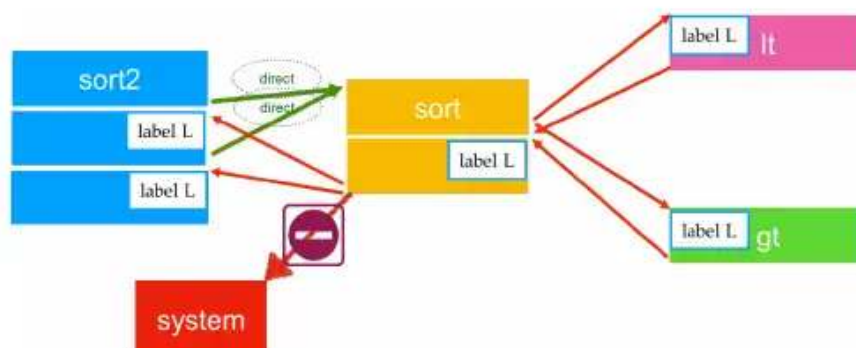
- Compute CFG: during **compilation** or by **static analysis** of resulting instructions
 - CFG captures expected behaviour
- **Monitor** behaviour during execution: ensure only paths in CFG are followed
 - **Direct calls** do not need to be monitored (assuming code is immutable)
 - **Indirect calls** (via saved values, based on registers, function tables, etc) need to be monitored
 - Depend on data that can be dynamically altered
 - Monitoring part called **In-line Reference Monitor (IRM)**

Security - Application Security II - Defense - prevent, detect and kill attacks - image-22.webp

- **Direct vs. Indirect Calls:** * **Direct call:** `myFunction()`. The destination address is hardcoded and immutable (cannot be changed). CFI doesn't need to monitor these.
 - **Indirect call:** `function_pointer()`. The destination is saved in a variable/register. Because variables can be overwritten by attackers, CFI *must* monitor these.
 - **How Labels Work:** The compiler inserts a secret ID (a label) at the beginning of every valid function. Before the program makes an indirect jump, the compiler inserts a check: "I am about to jump to Address X. Does Address X have the correct label?"

Detecting deviations

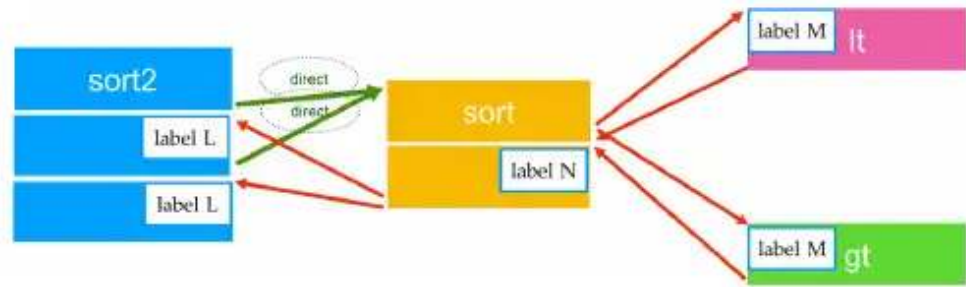
- Add instructions to the program to check that current control flow is expected
- Insert a **label** (canary value) before each possible **jump target**
 - In the code, before the first instruction of jump target
- Before jump, check that target address contains label or abort



Does not prevent reuse: can jump to any target that is valid *at least once*

Security - Application Security II - Defense - prevent, detect and kill attacks - image-23.webp

Detailed labelling



Constraints on labels:

- lt and gt both called from sort: must share a label
- Two possible return sites from sort: must share a label

⇒ This is starting to get difficult. . .

Security - Application Security II - Defense - prevent, detect and kill attacks - image-24.webp

- **Sharing Labels:** If `Sort` is allowed to jump to either `LT` (less than) or `GT` (greater than), then `LT` and `GT` must share the exact same label, so `Sort`'s check passes for both. That's tough because in a perfect world you want unique canaries?

instrumentation of a destination for jump: add label

Bytes (opcodes)	x86 assembly code	Comment
8B 44 24 04	mov eax, [esp+4]	; first instruction
...		; of destination code
can be instrumented as (a):		
78 56 34 12	DD 12345678h	; label ID, as data
8B 44 24 04	mov eax, [esp+4]	; destination instruction
...		

instrumentation of jump: check label

Bytes (opcodes)	x86 assembly code	Comment
FF E1	jmp ecx	; a computed jump instruction
can be instrumented as (a):		
81 39 78 56 34 12	cmp [ecx], 12345678h	; compare data at destination
75 13	jne error_label	; if not ID value, then fail
8D 49 04	lea ecx, [ecx+4]	; skip ID data at destination
FF E1	jmp ecx	; jump to destination code

Security - Application Security II - Defense - prevent, detect and kill attacks - image-26.webp

Possible attacks on CFI

- ☹️ Attacker injects code that has a legal label
 - 👍 Not possible for non-executable memory and non-writable code segments
- ☹️ Modify code labels to allow new control flows
 - 👍 Not possible for non-writable code segment
- ☹️ Make it look like a CFI check passed
 - 👍 CFI computations done in registers only: unavailable for attacker
 - 👍 No stack usage for CFI labels

Security - Application Security II - Defense - prevent, detect and kill attacks - image-27.webp

CFI caveats

- 👍 CFI defeats all attacks that change expected control flow: ROP, code injection, ...
- ☹️ CFI analysis and instrumentation can be heavy
 - Control flow analysis: very close to halting problem...
 - Check on "hot paths" (functions called many times) can eventually slow down program
- ☹️ Still allows control transfer that are compliant
 - **mimicry attacks**
 - Constraints for complex programs may limit actual label diversity
- ☹️ Does not prevent data leak or data corruption
 - In particular: read overflow, buffer overflow to affect control flow inside a basic block

Security - Application Security II - Defense - prevent, detect and kill attacks - image-28.webp

- **Halting Problem:** This is a famous computer science theorem stating you cannot perfectly predict what a program will do without running it. Therefore, building a 100% perfect CFG is mathematically impossible; the compiler has to make conservative guesses.
- **Hot Paths:** A "hot path" is a loop or function that runs millions of times a second. Adding a label check inside a hot path causes massive performance slowdowns.

- **Mimicry Attacks:** If the attacker redirects the flow to a malicious function, but that function *happens to share the same label* as the intended function, the CFI check passes. The attacker "mimicked" legitimate control flow.
-

Part 4: Memory Safety and Type Safety (Slides 27-34)

Defense 5: Memory Safety Languages

1. **Preventive:** Makes the exploit impossible to execute.
2. **Detective:** Notices the exploit is happening and kills the program.
3. **Reactive/Corrective:** Fixes the damage after the fact (we mostly focus on the first two).

Security - Application Security II - Defense - prevent, detect and kill attacks - image-29.webp

Category: **PREVENTIVE**

C and C++ are inherently unsafe. Languages like Python, Java, and Rust are Memory Safe.

Memory safety ensures programs only access authorized memory, divided into spatial (location) and temporal (time) safety. **Spatial safety prevents out-of-bounds access (e.g., buffer overflows).** **Temporal safety ensures memory is not used after it is deallocated (e.g., use-after-free).** Safe languages like Rust use compiler checks, while others like Java use garbage collection. 🇺🇸 University of Maryland +4

Spatial Memory Safety

Spatial safety concerns *where* a program reads or writes memory. A violation occurs when a program accesses memory outside the intended boundaries of an allocated object. 🇺🇸 Rutgers University +1

- **Buffer Overflows/Over-reads:** Accessing an array index outside its bounds, such as accessing `array[10]` when only 5 elements are allocated.
- **Pointer Arithmetic Errors:** Incorrectly calculating pointer addresses to point to invalid locations.
- **Uninitialized Memory:** Reading data before it has been written to. 🇺🇸 University of Maryland +1

Temporal Memory Safety

Temporal safety concerns *when* a program accesses memory relative to its lifetime, as discussed in. A violation occurs when a pointer is used to access memory that has not yet been allocated, has been deallocated, or is no longer valid (a "dangling pointer"). 🇮🇩 The Indonesian Journal of Computer S... +2

- **Use-after-free:** Accessing memory after it has been freed, which is a major security concern in C/C++.
- **Double-free:** Freeing the same memory address twice, which can corrupt memory management structures.
- **Stack-use-after-return:** Accessing a local variable's memory after its function has returned. 🇺🇸 University of Maryland +2

Memory safety

- A **programming language is memory-safe** if all possible programs written in the language are memory-safe
- A **program is memory-safe** if all possible executions are memory-safe

Memory-safe execution:

- **Temporal safety:** only access memory that is **defined** (active)
 - Memory is **undefined** if not yet allocated, uninitialized, deallocated (freed)
⇒ Only create pointers through standard means: `p = malloc()`, `p = &x`, ...
- **Spatial safety:** when using a pointer to access memory, only access memory that **belongs** to that pointer

⇒ Spatial safety ensure that access is to a legal region, temporal safety ensures that content is still valid

Security - Application Security II - Defense - prevent, detect and kill attacks - image-31.webp

- **Temporal Safety (Time):** Is the memory currently valid *right now*?
 - *Use-after-free:* You ask for memory (`malloc`), use it, and give it back (`free`). But you kept the pointer! If you try to use the pointer later, the memory might now belong to a password manager. This is a temporal violation because that memory isn't yours *anymore*.
 - *Function stack return:* If a function returns a pointer to its own local stack variable, that variable is destroyed the millisecond the function ends. Using it afterward is a temporal violation.
- **Spatial Safety (Space):** Are you staying inside your boundaries?
 - *Extended Pointers (p, b, e):* In a safe language, a pointer isn't just an address. It's a hidden struct containing `p` (current position), `b` (base/start), and `e` (extent/end). When you say `pointer[5]`, the language dynamically checks: "Is `p+5` greater than `e`?" If yes, it throws an error and refuses to do it. This makes buffer overflows impossible! `b - e` is like the domain.
 - **Integer Overflow Caveat:** In C, an integer overflow (like `65535 + 1` wrapping to `0`) is bad math, but it's not a memory violation by itself. It only becomes a memory vulnerability if you use that corrupted `0` to allocate a buffer, and then try to write 10 bytes into a 0-byte space.

Spatial safety

- “Extended” pointer representation (p, b, e)
 - p is actual pointer
 - b is **base** (i.e. start) of memory region that p can access
 - e is **extent** (i.e. end/size) of memory region that p can access
- Pointer arithmetic can change p , but does not change b and e
- Access allowed if and only if $b \leq p \leq e - \text{sizeof}(\text{typeof}(p))$

```
int x;  
int* y = &x; // p = addr(x), b = addr(x), e = addr(x)+4 (assuming 4-byte int)  
int* z = y+1; // p = addr(x)+4, b = addr(x), e = addr(x)+4  
*y = 42; // OK: addr(x) <= addr(x) <= addr(x)+4-4  
*z = 42; // ERROR: addr(x) <= addr(x)+4 > addr(x)+4-4
```

Security - Application Security II - Defense - prevent, detect and kill attacks - image-33.webp

```
char buf[4]; // p = addr(buf), b = addr(buf), e = addr(buf)+4  
y[2] = "A"; // OK: p = addr(buf)+2, addr(buf) <= addr(buf)+2 <= addr(buf)+4-1  
y[4] = "B"; // ERROR: p = addr(buf)+4, addr(buf) <= addr(buf)+4 > addr(buf)+4-1
```

Security - Application Security II - Defense - prevent, detect and kill attacks - image-34.webp

This code looks like absolute gibberish if you aren't familiar with C, so don't worry. C is notorious for having dense, confusing syntax for memory management.

Think of computer memory as a massive row of mailboxes, where each mailbox holds exactly one byte.

- **int x;** You are telling the computer: "Reserve a block of 4 mailboxes for me to store a whole number, and call this block **x**."
- **& (The Address-Of Operator):** **&x** means "Give me the exact mailbox number (the memory address) where **x** starts."
- **int* y (The Pointer):** The ***** in a declaration means "pointer". You are telling the computer: "Create a special variable called **y**. It will not hold a normal number; it will only hold the *memory address* of an integer."
 - So, **int* y = &x;** means "Store the address of **x** inside the pointer **y**."
 - *** (The Dereference Operator):**
When used *outside* of a declaration, ***** means "Go to the address".
 - ***y = 42;** means "Go to the address stored inside **y**, and shove the number 42 into that mailbox."

The Spatial Safety Formula

The slides define a formula to prove if a memory access is safe:

$$b \leq p \leq e - \text{sizeof}(\text{typeof}(p))$$

- **b (Base)**: The start of your legal memory block.
- **e (Extent)**: The end of your legal memory block.
- **p (Pointer)**: Where you are currently trying to write data.
- **sizeof(typeof(p))**: How big the item is that you are trying to write. An `int` is 4 bytes. A `char` is 1 byte.

Why the minus sign at the end?

Imagine you have a parking space that ends at the 10-meter mark (`e` = 10). You want to park a 4-meter long car (`int` = 4). You cannot start parking the car exactly at the 10-meter mark (`p` = 10), because the car will spill 4 meters into the street! You must start parking at the 6-meter mark or earlier ($10 - 4 = 6$).

Breaking Down Snippet 1 (The Integer)

```
int x;  
int* y = &x; // p = addr(x), b = addr(x), e = addr(x)+4 (assuming 4-byte int)  
int* z = y+1; // p = addr(x)+4, b = addr(x), e = addr(x)+4  
*y = 42; // OK: addr(x) <= addr(x) <= addr(x)+4-4  
*z = 42; // ERROR: addr(x) <= addr(x)+4 > addr(x)+4-4
```

```
char buf[4]; // p = addr(buf), b = addr(buf), e = addr(buf)+4  
y[2] = "A"; // OK: p = addr(buf)+2, addr(buf) <= addr(buf)+2 <= addr(buf)+4-1  
y[4] = "B"; // ERROR: p = addr(buf)+4, addr(buf) <= addr(buf)+4 > addr(buf)+4-1
```

Security - Application Security II - Defense - prevent, detect and kill attacks - image-36.webp

```
int x;  
int* y = &x;
```

`x` gets 4 bytes. Let's say it starts at memory address 100.

- **b (Base)**: 100
- **e (Extent)**: 104
- **p (Pointer `y`)**: 100
y = 100 (value of memory address of the integer x), which is p in the formula

```
int* z = y+1;
```

Pointer Arithmetic: In C, adding 1 to an `int` pointer does *not* move it forward by 1 byte. It moves it forward by *one whole integer* (4 bytes).

- `p` (Pointer `z`): 104.

```
*y = 42; // OK
*z = 42; // ERROR
```

- Writing to `y` is safe because you start writing 4 bytes at address 100. It perfectly fits inside the 100-104 boundary.
- Writing to `z` is an **ERROR** because `z` starts at 104. If you try to write a 4-byte integer starting at 104, you write into mailboxes 104, 105, 106, and 107. You just overwrote memory you don't own!

Breaking Down Snippet 2 (The Buffer)

```
int x;
int* y = &x; // p = addr(x), b = addr(x), e = addr(x)+4 (assuming 4-byte int)
int* z = y+1; // p = addr(x)+4, b = addr(x), e = addr(x)+4
*y = 42; // OK: addr(x) <= addr(x) <= addr(x)+4-4
*z = 42; // ERROR: addr(x) <= addr(x)+4 > addr(x)+4-4
```

```
char buf[4]; // p = addr(buf), b = addr(buf), e = addr(buf)+4
y[2] = "A"; // OK: p = addr(buf)+2, addr(buf) <= addr(buf)+2 <= addr(buf)+4-1
y[4] = "B"; // ERROR: p = addr(buf)+4, addr(buf) <= addr(buf)+4 > addr(buf)+4-1
```

Security - Application Security II - Defense - prevent, detect and kill attacks - image-36.webp

(Note: the slide has a typo. It declares `buf` but then uses `y`. We will assume `y` is a pointer to the memory addresses of `x` (which was an `int`.)

```
char buf[4];
```

This is a buffer of 4 characters. A character is exactly 1 byte.
Remember `int x` it starts at address 100. Base = 100, Extent = 104.

```
y[2] = "A"; // OK
```

`y[2]` means "jump forward 2 slots". Because `char` is 1 byte, this moves the pointer forward exactly 2 bytes.

Pointer = 102. You write 1 byte ("A"). It lands inside mailbox 102. Safe!

```
y[4] = "B"; // ERROR
```

`y[4]` moves the pointer forward 4 bytes.

Pointer = 104. You write 1 byte ("B"). It lands in mailbox 104. Since your legal memory ended *exactly* before 104, this is an out-of-bounds spatial safety violation!

Memory safety violations

- Buffer overflow is spatial safety violation: accessing memory beyond e
- Format string vulnerabilities can allow spatial safety violations
- Use-after-free is a temporal safety violation
- Holding a pointer pointing inside a function's stack after the function returns is a temporal safety violation
- Integer overflows do not violate memory safety *as long as they don't cause an illegal pointer to be made*

```
1 unsigned short x = 65535;
2 x = x+1; // Overflows and becomes 0
3 printf("%d\n",x); // OK: memory-safe
4 char* p = malloc(x); // Allocates size-0 buffer
5 p[65535] = "A"; // ERROR
```

Security - Application Security II - Defense - prevent, detect and kill attacks - image-37.webp

Memory-safe languages

- Many languages will take care of memory safety for you
 - Python, Java, C#, Rust, ...
 - Rust explicitly implements "borrow checker": checks temporal safety at compile-time by automatically marking pointers as invalid
- ⇒ Easiest way to avoid buffer overflow vulnerabilities
- Most of the time: add checks at runtime
- C and C++ are **not memory-safe**
- **C and C++ are not going away**
 - Although rustacean are trying really hard to make it so :)
- Possible to write memory-safe C/C++: compiler-added checks for memory violations
 - Not universally used due to performance penalties
 - Many ways to create a wrong pointer in C

Security - Application Security II - Defense - prevent, detect and kill attacks - image-38.webp

- **Rust's Borrow Checker:** Rust enforces temporal safety at compile-time. It tracks exactly who "owns" or is "borrowing" memory. If you try to compile code that might result in a use-after-free, the compiler literally refuses to compile the program.

Defense 6: Type Safety

Category: **PREVENTIVE**

Type safety

- Each object has a type: int, pointer to int, struct, pointer to function, etc
 - Implicit in some languages, but definitely there
- **Type safety** ensures that operations on objects are always compatible with types
 - Injecting shellcode in a buffer and jumping via a valid buffer pointer is *technically* memory-safe w.r.t. the pointer... but not type-safe!
- Dynamically-typed languages (e.g. Python) have additional metadata to identify the type of each object
 - If an operation is not possible for the **current** type, throw exception
 - Well-defined behaviour and thus type-safe

Security - Application Security II - Defense - prevent, detect and kill attacks - image-39.webp

- **Type Safety:** Ensures you don't treat an integer as a string, or a string as executable code. Like in the code from above, y pointed to an int, but we wrote chars into the memory.

- **The Shellcode Example:** If you inject shellcode into a buffer, the program just sees an array of characters (bytes). If you somehow trick the program into jumping there, that is a **Type Safety violation**, because the program is executing an Array as if it were a Function. Type-safe languages assign metadata to every object. If the CPU tries to execute an Array, the language throws a Type Exception because Arrays do not have the "executable" type.

Costs of safety

Type and memory safety enforcement is expensive computationally

- Garbage-collected languages avoid temporal violations, but require more memory (and time) than malloc/free
- Runtime bounds check take time on each access but avoid spatial violations
- Verifying violations at compile time removes runtime costs but increases compilation costs
 - Generally a good tradeoff → Rust

Security - Application Security II - Defense - prevent, detect and kill attacks - image-40.webp

References

<https://gemini.google.com/app/0b36b9bc69fa4646>

chap 6

lec7 - Applciation Secuirty II